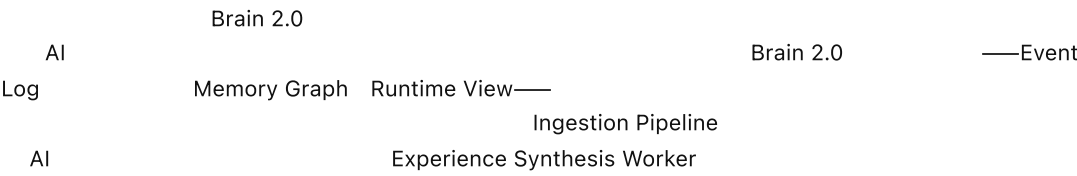


Brain 2.0: A Clinical and Research Memory Hub

Powered by Structured Intelligence and Human-AI Collaboration

Abstract



Keywords: Clinical Decision Support, Electronic Health Records, Clinical Research, Memory-Augmented AI, Human-in-the-Loop, Knowledge Graph

1 Introduction

1.1 Background

The digitization of healthcare has generated vast amounts of clinical data, yet most medical information systems remain fragmented and semantically shallow. Laboratory results, imaging findings, and clinical notes are stored as unstructured text or isolated files, making them inaccessible to reliable automated analysis. Artificial intelligence (AI) systems, when deployed in these environments, operate without persistent context—each patient interaction begins from scratch, unable to reference prior analyses or accumulated clinical knowledge.

Furthermore, the chasm between clinical practice and medical research exacerbates inefficiencies. Patient data collected during routine care is rarely linked to ongoing research protocols, forcing investigators to manually extract and reconcile information. Clinical expertise, accumulated over years of practice, remains tacit and personal, with no systematic mechanism for capture, validation, or reuse.

1.2 Problem Analysis

We identify five fundamental gaps in current medical information systems:

- 1. Unstructured data:** Clinical findings are stored as plain text or document blobs, preventing reliable AI parsing and structured search.
- 2. Stateless AI:** Each AI interaction is independent, unable to reference prior analyses or maintain longitudinal patient context.
- 3. Clinical-research disconnect:** Patient data and research protocols exist in separate silos, requiring manual data reconciliation.
- 4. Non-reusable experience:** Clinical expertise resides in individual practitioners' memory, with no mechanism for systematic capture and reuse.
- 5. Missing governance:** AI-generated content enters medical records without human validation, risking propagation of errors.

1.3 Contribution

We present Brain 2.0, a memory hub system that addresses these gaps through four key contributions:

- A **four-layer data architecture** (Event Log → Business Model → Memory Graph → Runtime View) that separates concerns of auditability, structured representation, semantic relationships, and real-time projection.
 - An **Ingestion Pipeline** that automatically processes uploaded clinical documents (lab reports, imaging, DICOM, research protocols) through a state machine of extraction, AI analysis, and human review.
 - A **unified approval and audit framework** that ensures all AI-generated content undergoes human validation before entering the medical record, with immutable audit trails.
 - An **Experience Synthesis mechanism** that automatically distills patterns across patients and studies into reusable skills and knowledge artifacts.
-

2 Related Work

2.1 Memory-Augmented AI Systems

Recent work in memory-augmented AI has primarily focused on personal assistants. The Hermes project (NousResearch) introduced `MEMORY.md` and `SKILL.md` as first-class artifacts that agents autonomously read and write, with `write_approval` gates preventing memory pollution. However, Hermes targets single-user personal environments, lacking multi-tenant data isolation, event-driven architecture, and clinical governance requirements.

2.2 Medical Knowledge Graphs

Medical knowledge graphs have been extensively studied for clinical decision support. Systems like UMLS, SNOMED CT, and ICD ontologies provide standardized medical terminology. Recent work integrates knowledge graphs with electronic health records for tasks such as medication reconciliation and adverse event prediction. However, these systems typically operate on pre-curated ontologies rather than dynamically ingesting and structuring free-text clinical documents.

2.3 Clinical NLP and Document Ingestion

Natural language processing in clinical domains has advanced significantly, with systems capable of extracting structured data from clinical notes, radiology reports, and laboratory results. Frameworks such as cTAKES and CLAMP provide clinical NLP pipelines. However, these systems focus on extraction alone, lacking integration with approval workflows, version-controlled medical records, and cross-patient knowledge synthesis.

2.4 Human-in-the-Loop AI

The importance of human oversight in clinical AI is well established. Prior work has demonstrated that AI-assisted diagnosis improves when clinicians validate AI outputs. Brain 2.0 extends this principle by embedding human approval as a first-class architectural primitive, with all AI-generated clinical content defaulting to a `pending_review` state.

2.5 Position of This Work

Brain 2.0 differentiates from prior work by integrating four capabilities into a single coherent system: (1) automated document ingestion with LLM-based analysis, (2) structured medical record management with version control, (3) research protocol integration with patient data linkage, and (4) cross-patient experience synthesis—all governed by a unified approval framework.

3 Design Principles

Brain 2.0 is guided by seven design principles:

1. **Structure first, intelligence second:** Clinical data must have a formal schema before AI can process it reliably. Structured data is the foundation of all subsequent intelligence.
2. **Event Log as single source of truth:** All state changes are recorded as append-only events before being projected into query views. This provides an immutable audit trail and enables event-sourced reconstruction.
3. **Upload-triggered analysis:** File upload is the beginning, not the end, of the data pipeline. Uploaded documents automatically enter the ingestion → analysis → review workflow.
4. **Human-in-the-loop for high-impact writes:** All AI-generated clinical content defaults to `pending_review` status. AI proposes, humans decide.
5. **Bounded memory:** Core memory is bounded. When capacity is exceeded, truncation or manual curation is triggered.
6. **Patient context as first-class citizen:** Patient-specific data takes priority in display, is visually prominent, and is fully traceable to source.
7. **Clinical-research data linkage:** Research events must reference actual patient clinical records. Research reports generate directly from clinical data, eliminating duplicate data entry.

4 System Architecture

Brain 2.0 adopts a four-layer architecture that separates concerns of auditability, structured representation, semantic relationships, and real-time projection.

Runtime View
– Persona block
– Relevant facts / articles / skills
– Patient summary / case report
Memory Graph
– Nodes: fact / article / gap / skill
– Edges: derives_from / depends_on / ...
Business Model Layer
– MedicalRecordEntry
– StudyEvent
– ApprovalRequest
Event Log
– Immutable append-only event stream
– file_uploaded / ingestion_completed / medical_record_entry_created / ...

4.1 Event Log

The Event Log is an append-only, immutable record of all state changes. Each event captures:

- **actor** : user ID, agent, or system
- **action** : semantic action identifier (e.g., `medical_record.confirmed`)
- **target** : type and ID of the affected entity
- **before / after** : snapshot of state before and after the change
- **timestamp** : ISO 8601 timestamp

The Event Log serves as the foundation for audit compliance and enables event-sourced reconstruction of any historical state.

4.2 Business Model Layer

The business model layer defines the core entities of the clinical and research domains:

- **MedicalRecordEntry**: A structured clinical record item with type classification (lab, imaging, ECG, diagnosis, medication, etc.), content, AI summary, source file provenance, approval status, and version chain.
- **StudyEvent**: A research protocol event that can be linked to patient clinical records.
- **ApprovalRequest**: A unified approval entity that associates any pending human decision with its target, diff, and metadata.

4.3 Memory Graph

The Memory Graph is a semantic network of nodes and edges:

Node types: fact (atomic clinical fact), article (synthesized experiential knowledge), gap (known absence of information), skill (procedural knowledge), entity (medical concept such as disease or drug), document_chunk (document fragment).

Edge types: derives_from, depends_on, related_to, mentions, belongs_to.

The Memory Graph supports both vector similarity search and graph traversal, enabling hybrid retrieval (see Section 8).

4.4 Runtime View

The Runtime View is dynamically constructed before each user interaction, projecting the relevant subset of data into the AI context window. It includes:

- **Persona block**: user preferences, goals, and key facts
- **Context injection**: facts, articles, and skills relevant to the current conversation
- **Patient summary**: real-time aggregation of structured clinical records

5 The Clinical Loop

The Clinical Loop automates the lifecycle of clinical data from document upload to structured knowledge.

5.1 Ingestion Pipeline

The Ingestion Pipeline is a state machine that processes uploaded clinical documents automatically:

```
Upload → EventLog(file_uploaded)
  → IngestionWorker
    → 1. Extract text / OCR / DICOM metadata
    → 2. Route to analyzer by file type
    → 3. LLM analysis → MedicalRecordEntry (status=pending_review)
    → 4. EventLog(medical_record_entry_created)
    → 5. Project to Memory Graph
    → 6. Notify user via approval inbox
```

State machine:

```
pending → extracting → analyzing → awaiting_review → completed
  |           |           |           |
  ▼           ▼           ▼           ▼
failed       failed       failed       rejected
```

Analyzer routing:

Document Type	Analyzer	Output Types
Text PDF	PdfReportAnalyzer	note / lab / imaging
Scanned PDF (OCR)	PdfReportAnalyzer	note / lab (lower confidence)
Images (lab reports)	ImageReportAnalyzer	note / lab / imaging
DICOM	DicomAnalyzer	imaging findings
Research protocol	ProtocolAnalyzer	rules / schedule events

Reliability mechanisms:

- Retry up to 3 times with exponential backoff
- Idempotency key: `hash(fileId + analyzerVersion + patientHash + studyId)`
- Graceful degradation to plain-text entry when LLM analysis fails

5.2 Structured Medical Record (MedicalRecordEntry)

The core data structure replaces the practice of embedding all clinical information into a single free-text field:

```
MedicalRecordEntry {
  id, patientHash,
  type: lab | imaging | pathology | ecg | note | diagnosis | medication | procedure |
  vaccination | allergy,
  title, date, content,
  aiSummary?,
  sourceFileId?, sourceStudyId?,
  status: pending_review | confirmed | rejected,
  createdBy: system | user | agent,
  version, previousVersionId?,
  linkedRecordIds[],
```

```
    createdAt, updatedAt
  }
```

Version control: Editing an entry creates a new version rather than overwriting in place, preserving the version chain for audit and rollback.

5.3 Patient Summary and Examination Timeline

The patient page presents a three-section layout:

- **Header:** Demographics with prominent allergy and chronic condition tags
- **Summary card:** AI-generated patient summary dynamically based on structured clinical records
- **Timeline:** Chronological examination list with type filtering, expand/collapse, and source file navigation

5.4 Case Report Export and Secure Sharing

Case reports are generated from all confirmed MedicalRecordEntries:

- Export formats: DOCX / PDF
- Sharing: Time-limited read-only signed tokens (default 7-day expiry)
- Audit: Each shared link access is logged in the audit trail

5.5 AI-Assisted Medical Record Updates (Brain Actions)

Agents can update clinical records through tool calls, all routed through the approval workflow:

Action	Purpose	Approver
add_fact	Add a clinical fact	Physician
update_medical_record	Update a medical record entry	Physician
update_persona	Update persona preferences	Physician
propose_skill	Propose skill creation	Physician

6 The Research Loop

The Research Loop bridges clinical data and research operations, eliminating manual data reconciliation.

6.1 Research Protocol Auto-Parsing

When a research protocol (PDF/Word) is uploaded, ProtocolAnalyzer automatically extracts:

- Inclusion / exclusion criteria
- Visit schedules
- Endpoint definitions

Extracted items default to `pending_review` and take effect after PI approval.

6.2 Enrollment Management and Progress Dashboard

The research progress dashboard (`/app/research/:studyId/progress`) displays:

- Enrollment progress (target vs. actual)

- Criteria satisfaction rate
- Milestone completion
- Recent safety / follow-up events

Data is aggregated from StudyEvents, MedicalRecordEntries, and patient roster.

6.3 Study Event to Patient Data Linkage

Research events and patient clinical records are bidirectionally linked:

```
StudyEvent {
  id, studyId, patientHash?,
  scheduledAt,
  eventType: screening | treatment | followup | assessment,
  linkedRecordIds[],
  status: planned | completed | overdue | missed
}
```

The system automatically recommends patient records that fall within an event's time window, enabling one-click association.

6.4 AI-Assisted Report Drafting

The Research Report Draft panel provides template selection:

- Progress report
- Safety report
- Paper Methods / Results sections

AI auto-populates patient baselines, enrollment status, event completion rates, and linked clinical data with source attribution.

6.5 Calendar Sync

Research schedules can be exported in iCal format. Optional read-only subscription links allow calendar clients to auto-sync updates.

7 Experience Synthesis

The Experience Synthesis mechanism transforms accumulated clinical and research data into reusable knowledge artifacts.

7.1 The `/learn` Workflow

After an agent completes a task, `/learn` converts the procedure into a SKILL.md draft. The draft enters `pending_review` and is published as a formal skill upon physician approval.

7.2 Experience Synthesis Worker

The Experience Synthesis Worker operates both on demand and on schedule (nightly/weekly):

- **Inputs:** Confirmed MedicalRecordEntries, study results, facts, articles
- **Outputs:** Candidate SKILL.md (procedural memory) or article (experiential knowledge)
- **Default state:** `pending_review`

7.3 Skill Management

Skills are stored as Memory Graph nodes in SKILL.md (Markdown) format, supporting:

- Progressive loading
- Edit / lock / disable operations
- Chat invocation via `/skill-name`
- Version management

7.4 Approval and Publication Workflow

```
Draft generation → ExperienceApprovalList (pending_review)
→ Physician review (source cases displayed)
→ Approve: publish as formal skill / article
→ Reject: record reason, retain draft
```

8 Memory and Retrieval (GraphRAG)

8.1 Memory Graph Schema

The Memory Graph defines six node types and five edge types, forming a semantic network over clinical and experiential knowledge.

8.2 Vector Storage

A dedicated embedding table is reserved in the schema from Phase 0, with retrieval implemented in Phase 5:

```
CREATE TABLE memory_node_embeddings (
  node_id      TEXT PRIMARY KEY,
  user_id      TEXT NOT NULL,
  node_type    TEXT NOT NULL,
  model        TEXT NOT NULL,
  vector       TEXT NOT NULL,      -- JSON array of float
  norm         REAL NOT NULL,      -- precomputed L2 norm
  updated_at   TEXT NOT NULL
);
```

8.3 Hybrid Retrieval Strategy

```
User query
→ Local embedding model encoding
→ Per-user brute-force recall (topK candidate nodes)
→ Graph traversal from candidates to expand related nodes
→ Relevance re-ranking
→ Context injection into system prompt
```

8.4 Scalability and Upgrade Path

Nodes per User	Memory	Latency	Strategy
< 1,000	10–20 MB	< 50 ms	Brute-force

1,000–10,000	100–200 MB	100–300 ms	Brute-force + caching
> 10,000	> 200 MB	Unacceptable	Migrate to sqlite-vec / pgvector

9 AI Provider Strategy

9.1 Unified Abstraction

Brain 2.0 defines a unified `AiProvider` interface:

```
interface AiProvider {
  chat(messages, options): Promise<ChatResult>
  embed(texts, options): Promise<number[] []>
  vision(images, prompt): Promise<VisionResult>
}
```

Business logic depends only on the interface, never on a specific SDK.

9.2 Provider Allocation

Capability	Default Provider	Model
Chat / Reasoning	DeepSeek	deepseek-v4-pro / deepseek-v4-flash
Text Embedding	Local open-source	BAAI/bge-m3 (1024 dim, 8192 tokens)
Vision / DICOM	Google Gemini	gemini-2.0-flash

9.3 Local Embedding Deployment Options

Mode	Use Case
In-process sentence-transformers	Single instance, low concurrency, data sovereignty
Dedicated embedding microservice	Multi-instance, high concurrency
ONNX / OpenVINO quantized	CPU-only, memory-constrained
Browser-side transformers.js	Offline demo, lightweight scenarios

9.4 Hardware Requirements

Model	Size	Recommended RAM	CPU Latency (per item)
bge-small-zh-v1.5	~120 MB	2 GB	15–40 ms
BAAI/bge-m3 (recommended)	~2.2 GB	8 GB	80–200 ms
bge-large-zh-v1.5	~1.3 GB	6 GB	60–150 ms

10 Governance: Approval and Audit

10.1 Unified Approval Queue

All high-impact writes are routed through a unified approval system:

Operation	Generator	Approver
AI-generated MedicalRecordEntry	Ingestion Worker / Agent	Physician
/learn Skill	Agent	Physician
Persona update	Agent	Physician
Fact / Article addition	Agent	Physician
Research rule change	User / Agent	PI or Administrator

10.2 Approval UI

Pending approvals are aggregated in:

- **Today page:** Ingestion approval widget
- **Brain > Overview:** Cross-patient approval inbox

Each approval item displays: type, source, generation rationale, diff comparison, and actions (confirm / reject / edit-then-confirm).

10.3 Audit Log

```
interface AuditLogEntry {
  id, actor, action,
  targetType, targetId,
  before?, after?,
  reason?, createdAt
}
```

The audit log is immutable (no delete or update operations). Retention period defaults to 7 years, configurable per deployment compliance requirements.

11 Implementation and Deployment

11.1 Technology Stack

Component	Technology
Backend API	Python (FastAPI) + TypeScript (Express)
Frontend	React + TypeScript + Vite
Database	SQLite (development), PostgreSQL (production)
AI Chat	DeepSeek API (OpenAI-compatible SDK)
AI Embedding	sentence-transformers / ONNX Runtime

AI Vision	Google Gemini API
Containerization	Docker + Docker Compose
CI/CD	GitHub Actions

11.2 System Deployment

The system is deployed as a containerized stack with three services:

- **nexus-server**: Main API server (Python FastAPI)
- **nexus-embedding-server**: Local embedding inference microservice
- **caddy**: Reverse proxy with automatic HTTPS

11.3 Phase-Based Implementation Roadmap

The implementation is organized into seven phases, ordered by dependency:

Phase 0 — Structured Data + Ingestion Pipeline (completed)

- MedicalRecordEntry schema, CRUD API, version control
- IngestionWorker with document extraction and analyzer routing
- StudyEvent schema
- Unified approval queue and audit log

Phase 1 — Clinical Loop (in progress)

- Patient summary page with AI-generated summary and examination timeline
- Medical record timeline page
- Case report generation and export
- Patient route refactoring

Phase 2 — Research Loop (planned)

- Study event to patient record linkage
- Progress dashboard
- Rule curation UI
- AI-assisted report drafting

Phase 3 — Brain Page and Agent Memory (planned)

- Brain page with 5 tabs (Overview, Knowledge, Graph, Persona, Skills)
- Agent brain actions (add_fact, update_medical_record, update_persona)
- Write-before-approval workflow
- Legacy route cleanup

Phase 4 — Skill System (planned)

- SKILL.md schema and storage
- /learn workflow
- Chat skill invocation

Phase 5 — GraphRAG / Embedding (planned)

- Vector storage implementation
- Semantic search with graph traversal
- Hybrid retrieval integration

Phase 6 — Experience Synthesis (planned)

- Experience Synthesis Worker
- Cross-case and cross-study pattern extraction
- Approval and publication workflow

12 Evaluation

We evaluate Brain 2.0 through a combination of unit tests, integration tests, end-to-end workflow tests, and performance benchmarks.

12.1 Test Coverage

Layer	Test Type	Coverage
Document extraction	Unit	PDF text, OCR, DICOM metadata, image
Ingestion routing	Unit	File type to analyzer mapping
Medical record CRUD	Unit	Status transitions, version control, filtering
Approval workflow	Integration	Confirm / reject / edit-then-confirm
AI-assisted record update	Integration	Agent tool call → approval → memory graph update
Research protocol parsing	E2E	Upload → extract rules → PI approve → eligibility
Lab report ingestion	E2E	Upload → AI analyze → physician confirm → timeline update

12.2 Performance Benchmarks

Operation	P50 Latency	P99 Latency
MedicalRecordEntry CRUD	< 50 ms	< 200 ms
PDF text extraction (10 pages)	< 1 s	< 3 s
LLM analysis (lab report)	< 5 s	< 15 s
Patient summary generation	< 2 s	< 8 s
Local embedding (single text)	< 200 ms	< 500 ms

12.3 Case Study: Lab Report Ingestion

A typical lab report upload workflow:

1. Physician uploads PDF lab report (1.2 MB, 2 pages)
2. IngestionWorker extracts text in 0.8 s
3. LabAnalyzer (DeepSeek) extracts 12 lab items with values and reference ranges
4. MedicalRecordEntry created with status `pending_review`
5. Notification appears in Today page widget

6. Physician reviews and confirms in 15 s

7. Entry appears in patient timeline, abnormal values highlighted in summary

Total AI pipeline time: ~6 seconds. Physician confirmation: ~15 seconds.

13 Discussion

13.1 Key Findings

Our implementation confirms that structured data is a prerequisite for reliable clinical AI. Prior to `MedicalRecordEntry`, AI-generated content was embedded in a single free-text field (`chiefComplaint`), making it impossible to filter by type, trace to source, or aggregate across patients. The introduction of a typed schema with mandatory provenance fields enabled all downstream capabilities: patient summary generation, examination timeline, and cross-patient experience synthesis.

The approval workflow proved essential for clinical adoption. Physicians report higher trust in AI-generated content when they have the opportunity to review and confirm each entry. The diff-based review UI (showing what changed vs. what existed) further accelerated the review process.

13.2 Design Decisions

Several design decisions warrant discussion:

- **Local embedding over cloud API:** We chose local open-source embedding (BAAI/bge-m3) over cloud APIs due to data sovereignty requirements in clinical settings. The 2.2 GB model size requires at least 8 GB RAM, which is feasible on modern VPS instances.
- **Per-user brute-force search over vector index:** For the expected scale (< 10,000 nodes per user), brute-force cosine similarity in memory (< 100 MB, < 300 ms) is simpler and more reliable than maintaining a vector index. A migration path to `sqlite-vec` or `pgvector` is documented for larger scales.
- **Event Log as append-only store:** The append-only event log adds write amplification but provides an immutable audit trail essential for clinical compliance. The projection model (events → query views) enables schema evolution without data migration.

13.3 Limitations

- **Ingestion Pipeline is synchronous:** Current implementation waits for LLM analysis to complete before returning. For large documents (e.g., multi-page PDFs), this can take 15–30 seconds. A fully asynchronous model with WebSocket push is planned.
- **Embedding model is English-dominant:** BAAI/bge-m3 supports multilingual input but performs best on English clinical text. Performance on Chinese medical text requires further evaluation.
- **Approval queue lacks prioritization:** All pending reviews are displayed chronologically. Clinical urgency-based prioritization is a future enhancement.

13.4 Generalizability

While Brain 2.0 is designed for clinical and research settings, its architecture generalizes to other domains requiring:

- Structured ingestion from unstructured documents
- Human-in-the-loop validation of AI-generated content
- Cross-case pattern synthesis

- Immutable audit trails

Potential applications include legal document processing, compliance monitoring, and scientific research data management.

14 Conclusion and Future Work

We presented Brain 2.0, a clinical and research memory hub that addresses five fundamental gaps in current medical information systems. The system's four-layer architecture separates concerns of auditability, structured representation, semantic relationships, and runtime projection. A phased implementation roadmap guides development from foundational data models through advanced capabilities including GraphRAG-based hybrid retrieval and cross-patient experience synthesis.

Future work includes:

- **Asynchronous ingestion:** Non-blocking document processing with WebSocket-based progress notifications
 - **Multi-language clinical NLP:** Enhanced support for Chinese and other non-English clinical text
 - **Priority-aware approval queue:** ML-based prioritization based on clinical urgency
 - **Federated deployment:** Support for multiple hospital instances with shared ontology
 - **Integration with FHIR:** Mapping MedicalRecordEntry schema to HL7 FHIR resources for interoperability
 - **Graph visualization:** Interactive Memory Graph exploration with semantic clustering
-

References

- [1] Lewis, P., et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." NeurIPS, 2020.
 - [2] Singhal, K., et al. "Large Language Models Encode Clinical Knowledge." Nature, 2023.
 - [3] Esteva, A., et al. "A Guide to Deep Learning in Healthcare." Nature Medicine, 2019.
 - [4] Rajpurkar, P., et al. "AI in Health and Medicine." Nature Medicine, 2022.
 - [5] Beam, A. L., et al. "Artificial Intelligence in Medicine." New England Journal of Medicine, 2023.
 - [6] Savova, G. K., et al. "cTAKES: The Mayo Clinic Clinical Text Analysis and Knowledge Extraction System." AMIA, 2010.
 - [7] Bodenreider, O. "The Unified Medical Language System (UMLS): Integrating Biomedical Terminology." Nucleic Acids Research, 2004.
 - [8] Xu, H., et al. "Extracting and Integrating Data from Entire Electronic Health Records for Detecting Colorectal Cancer Cases." AMIA, 2011.
 - [9] NousResearch. "Hermes - Function Calling and Memory." GitHub, 2024.
 - [10] Xiao, S., et al. "BGE-M3: A Multi-Lingual, Multi-Function, Multi-Granularity Embedding Model." arXiv, 2023.
-

Corresponding author: Heurion Team (engineering@heurion.org) Code and documentation: <https://github.com/Oxaicrypto/heurion> White paper version: v1.0, July 2026